جامعة الشرقية
A' SHARQIYAH UNIVERSITY

INTE130 – Object-Oriented Programming Assignment.

(CyberShield system Project - Implementing the system.)

Sumitted Parts: Part D

Prepared by :

Sultan Khalid Al Yarabi - 2318592

Malik Marzooq Al Rahbi - 2527871

Khamis Hamed Alowaisi - 2422670

Mohammed Naeem Al Gheilani – 2526168

Instructor Name:

Dr. Akbar khanan

Semester: Spring 26

1. Introduction :

In this part, the CyberShield system is implemented using Python language and based on the concept of Object-Oriented Programming (OOP). This system represents a simple one-person cybersecurity company, where the founder manages the main task with the help of three digital agents. Each agent has a specific job that helps analyze the problem and deliver the result, and then the founder makes the final decision.

2. The idea of implementing the system:

The implementation of the system depends on the presence of a main task created by the founder, which is checking the security of the system. Next, this task is passed to three agents:

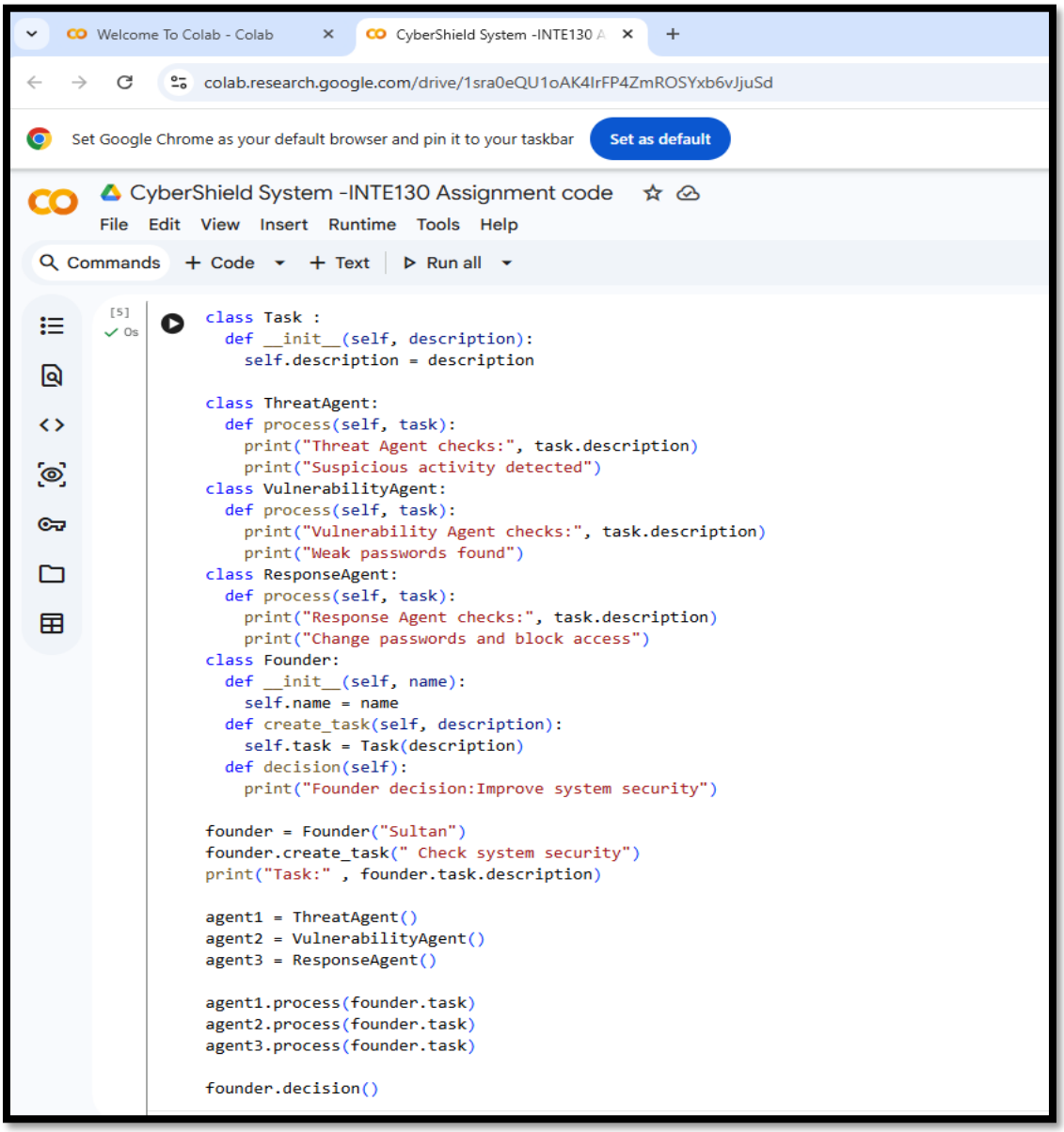
1- **Threat Agent:** Detects suspicious activity.

2- **Vulnerability Agent:** identifies vulnerabilities.

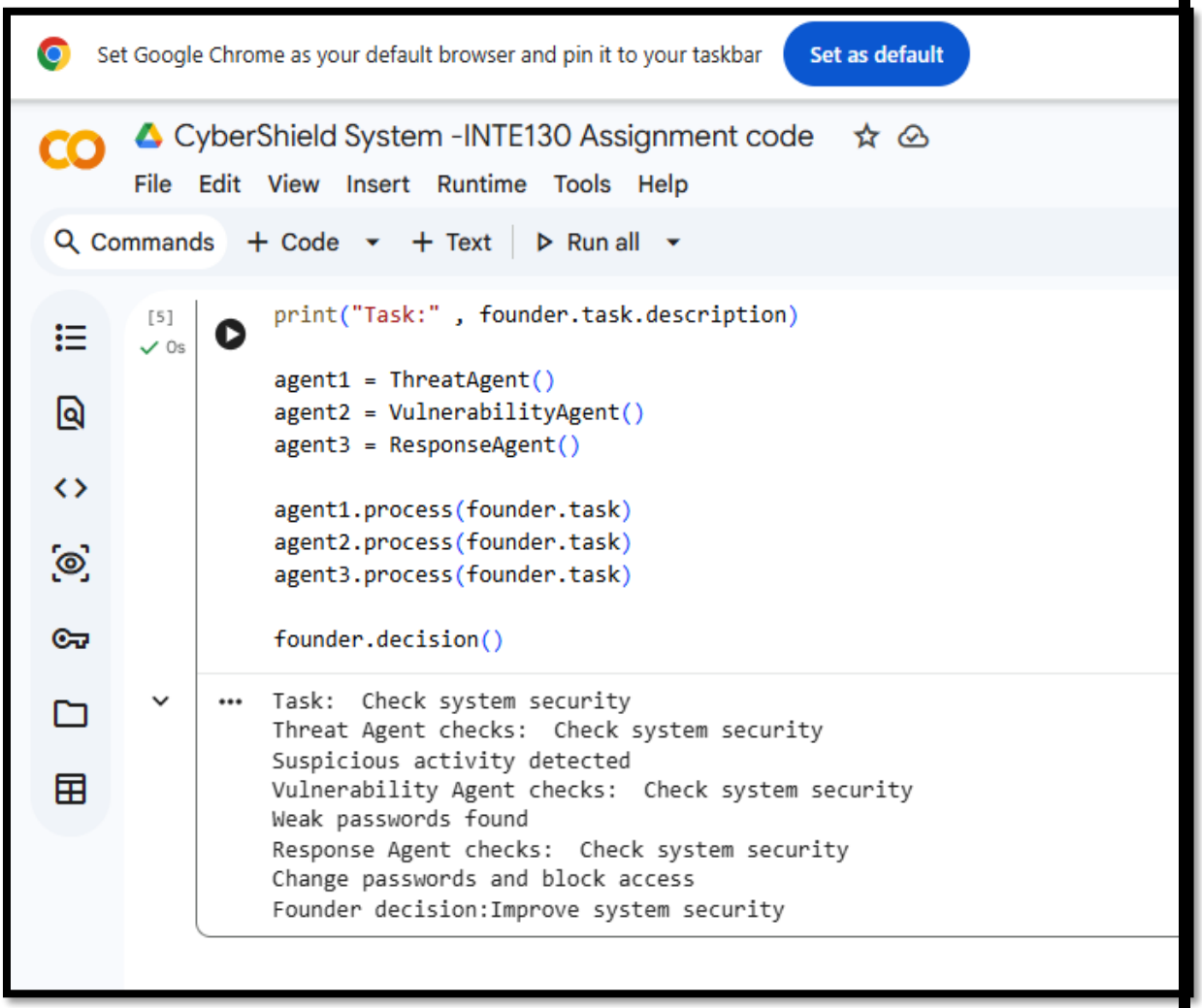
3- **Response Agent:** Suggests the appropriate solution.

❖ Finally, the Founder reviews the results and makes the final decision.

3- System code :



4 – Output:


Set Google Chrome as your default browser and pin it to your taskbar
Set as default
CyberShield System -INTE130 Assignment code
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[5]
0s
print("Task:" , founder.task.description)
agent1 = ThreatAgent()
agent2 = VulnerabilityAgent()
agent3 = ResponseAgent()
agent1.process(founder.task)
agent2.process(founder.task)
agent3.process(founder.task)
founder.decision()
Task: Check system security
Threat Agent checks: Check system security
Suspicious activity detected
Vulnerability Agent checks: Check system security
Weak passwords found
Response Agent checks: Check system security
Change passwords and block access
Founder decision:Improve system security

5 - Explanation of the code:

The program begins by creating a Task class that represents the task to be performed. After that, three classes representing digital agents were created: ThreatAgent, VulnerabilityAgent, and ResponseAgent. Each agent contains a process function that processes the task and displays its result.

The Founder class has also been created, which represents the founder, and contains a decision function to display the final decision. In the main part of the program, the founder creates the task, which is then passed to the three agents one by one. After each agent presents its score, the founder makes the final decision to improve the security of the system.

5. Explanation of the code:

The program begins by creating a Task class that represents the task to be performed. After that, three classes representing digital agents were created: ThreatAgent, VulnerabilityAgent, and ResponseAgent. Each agent contains a process function that processes the task and displays its result.

The Founder class has also been created, which represents the founder, and contains a decision function to display the final decision. In the main part of the program, the founder creates the task, which is then passed to the three agents one by one. After each agent presents its score, the founder makes the final decision to improve the security of the system.

6. How does the code meet Part D requirements?

This code fulfills the requirements of Part D because it contains:

Founder object → **founder object**

At least three digital agents → **Three digital agents**

Tasks created by the Founder → **Task created**

Agent processing tasks → **Each agent processed the task**

Agent recommendations → **Each agent displayed a result or recommendation**

Founder final decision → **The founder made the final decision**

7. Object-oriented programming concepts used:

Several OOP concepts are implemented in this code, namely:

Classes: such as Founder, Task, and Agents

Objects: such as founder, task, and agent1

Methods: such as process and decision

Encapsulation: Each class contains its own data and functions

Abstraction: Each agent performs its function in a simple and clear way

8. Conclusion:

This part shows how a simple cybersecurity management system can be implemented using Python and OOP. It also shows how multiple digital agents can help a founder execute tasks and make the final decision in an organized and easy way.